Отчет о выполнении лабораторной работы по pthread

Антонов Артем Максимович 23КНТ3

October 22, 2026

1 Множество Мондельброта

1.1 Математический алгоритм

Множество Мандельброта — это совокупность всех чисел $c \in \mathbb{C}$, для которых последовательность

$$z_0 = 0, \qquad z_{n+1} = z_n^2 + c \tag{1}$$

является ограниченной для всех значений $n \in \mathbb{N}$. Иными словами, число c принадлежит множеству Мандельброта тогда и только тогда, когда

$$(\exists A \in \mathbb{R})\ (\forall n \in \mathbb{N})\ \ |z_n| < A. \tag{2}$$

Представим z^2 как комплексное число, возведенное в квадрат:

$$z = x + yi \tag{3}$$
$$z^2 = (x+yi)^2 = x^2 + 2xyi + y^2 i^2 = x^2 + 2xyi - y^2 = (x^2 - y^2) + 2xyi \tag{4}$$

Прибавим к получившемуся выражению $c = a + bi$

$$z^2 = (x^2 - y^2) + 2xyi + a + bi = (x^2 - y^2 + a) + (2xyb)i \tag{5}$$

В итоге получаем

$$z_{n+1} = (x^2 - y^2 + a) + (2xyb)i \tag{6}$$

Отсюда,

$$x_{n+1} = x_n^2 - y_n^2 + a \tag{7}$$
$$y_{n+1} = 2x_n y_n b \tag{8}$$

Теперь рассмотрим условие принадлежности числа c к множеству Мондельброта:

$$|z_n| \leq 2 \tag{9}$$

Так как модуль комплексного числа это расстояние от точки до начала координат, можно z представить так:

$$z_n = \sqrt{x_n^2 + y_n^2} \tag{10}$$

Далее подставим это в исходное выражение и возведемв квадрат

$$\sqrt{x_n^2 + y_n^2} < 2 \tag{11}$$
$$x_n^2 + y_n^2 < 4 \tag{12}$$

Итого, получаем, что при $x_n^2 + y_n^2 < 4$ точка c принадлежит множеству Мондельброта.

Соответственно, при $x_n^2 + y_n^2 > 4$ - точка c гарантированно не принадлежит множеству Мондельброта.

В итоге всех преобразований мы получили 3 основные формулы, которые будут использоваться в программе.

Для нахождения новый x и y на каждой итерации цикла проверки числа $c = a + bi$:

$$x_{n+1} = x_n^2 - y_n^2 + a \tag{13}$$
$$y_{n+1} = 2x_n y_n b \tag{14}$$

И условие выхода из цикла проверки числа c:

$$x_n^2 + y_n^2 > 4 \tag{15}$$

1.2 Обзор кода

Программа реализует следующий алгоритм нахождения точки множества Мондельброта:

1. С помощью генератора псевдослучайных чисел генерируем a и b и подгоняем их в диапазон точек.

2. Итерируемся N раз, на каждой итерации генерируя x и y по формулам, которые были выведены в прошлой главе

3. Проверяем условия выхода из цикла, полученное в прошлой главе, если оно выполняется - ставим флаг и break

4. Проверяем флаг, если мы видим значение, означающее, что мы не вышли из цикла с помощью break (0), то записываем точку в глобальных массив точек.

Этот алгоритм выполняется каждым потоком. Поток находит свое определенное количество точек, которое расчитывается по следующей формуле:

```
int n_points = npoints / nthreads + (my_rank < npoints % nthreads ? 1 : 0);
```

1.3 Замеры производительности

Замеры выполнялись на ubuntu 22.04 под WSL2. Процессор Intel Core i5-10400F - 6 ядер и 12 потоков.

Количество итераций N в каждом запуске - 1000. Выполнялось по 15 запусков на 1, 2, 4, 8, 12, 16, 24 потоках. Количество точек, которое необходимо найти фиксированное - 1000000.

Table 1: Результаты замеров

Run number	NPOINTS	Threads count	AVG	MIN
15	1 000 000	1	6.3862	6.2632
15	1 000 000	2	3.2562	3.1483
15	1 000 000	4	1.7345	1.6990
15	1 000 000	8	0.9184	0.9019
15	1 000 000	12	0.7091	0.6618
15	1 000 000	16	0.7943	0.7655
15	1 000 000	24	0.6934	0.6736

2 Своя реализация `rwlock`

2.1 Обзор кода

В рамках лабораторной работы была реализована структура `my_rwlock_t` и основные ее функции `rwlock`:

- `int my_rwlock_init(my_rwlock_t* loc)`
- `int my_rwlock_destroy(my_rwlock_t* loc)`
- `int my_rwlock_rdlock(my_rwlock_t* loc)`

- `int my_rwlock_rwlock(my_rwlock_t* loc)`
- `int my_rwlock_unlock(my_rwlock_t* loc)`

2.1.1 `my_rwlock_t`

Структура `my_rwlock_t` содержит в себе следующие поля:

- `pthread_mutex_t mutex` - мьютекс для внутренний операций.
- `pthread_cond_t readers_cond` - условная переменная для читателей.
- `pthread_cond_t writer_cond` - условная переменная для писателей.
- `int readers_counter` - счетчик активных читателей.
- `int wait_read_counter` - счетчик ожидающих читателей.
- `int wait_write_counter` - счетчик ожидающих писателей.
- `int writer_active` - флаг, показывающий, есть ли активный писатель.

2.1.2 `my_rwlock_init`

Функция `my_rwlock_init` инициализирует поля структуры с помощью соответствующих стандартных функций, при ошибке инициализации возвращает код ошибки.

2.1.3 `my_rwlock_destroy`

Вызывает соответствующие `destroy` функции для объектов структур `pthread_mutex_t` и `pthread_cond_t`

2.1.4 `my_rwlock_rdlock`

Функция реализует механизм блокировки читателей. Для любых операций с внутренними переменными используются мьютекс. Соответственно, в начале функции мы блокируем мьютекс. Далее мы увеличиваем каунтер ожидающих читателей и запускаем цикл while с проверкой того, есть ли активный писать или есть ли ожидающие писатели, если условие выполняется - усыпляем поток, иначе уменьшаем количество ожидающих читателей и увеличиваем количество читателей и разблокируем мьютекс.

2.1.5 `my_rwlock_wrlock`

Функция реализует механизм блокировки писателей. Общая идея та же, мы увеличиваем каунтер и заходим в цикл, где условием является наличие активных писателей или наличие читателей. Если условие выполняется - засыпаем, иначе ставим флаг активного писателя и уменьшаем каунтер ожидающих писателей

2.1.6 `my_rwlock_unlock`

Общая функция разблокировки. Визуально поделена на 2 блока - в первом мы изменяем переменные, соответствующие тому типа потока, который выходит, если это писать (смотрим по переменной `writer_active`), то убираем активного писателя, иначе уменьшаем количество читателей. Следующий блок отвечает за передачу эстафеты, если у нас есть ожидающие писатели, и при этом нет активного писателя и количество читателей равно 0, то посылаем сигнал спящему потоку-писателю, если ожидающих писателей нет - будим всех читателей.

2.2 Замеры производительности

Замеры выполнялись на ubuntu 22.04 под WSL2. Процессор Intel Core i5-10400F - 6 ядер и 12 потоков. STDLIB - программа с использованием `rwlock` из pthreads, LAB - своя реализация. Общие параметры для всех запусков: размер списка - 1000 элементов, количество операций - 80000 По 10 запусков на каждой тестовой конфигурации. Бралось минимальное и среднее время среди 10 запусков.

Table 2: Результаты замеров

Search / insert / delete, %	Threads	STDLIB		LAB	
		min	average	min	average
99.9 / 0.05 / 0.05	1	0.0474	0.0493	0.0494	0.0527
	2	0.0280	0.0289	0.0353	0.0363
	4	0.0225	0.0233	0.0366	0.0382
	8	0.0239	0.0257	0.0443	0.0457
	16	0.0233	0.0245	0.0548	0.0565
	24	0.0232	0.0240	0.0648	0.0661
80 / 10 / 10	1	0.9064	1.0115	0.9578	1.0231
	2	1.4652	1.4985	1.1762	1.2425
	4	1.5506	1.5791	1.6709	1.6909
	8	1.7337	1.7689	2.2691	2.2894
	16	2.2558	2.3096	3.0469	3.0712
	24	2.8347	2.8930	3.6787	3.7191
50 / 25 / 25	1	3.5385	3.5817	3.5746	3.6058
	2	4.9537	4.9961	4.1156	4.1923
	4	5.0197	5.0684	4.7311	4.7634
	8	5.2214	5.3020	5.2638	5.3256
	16	6.9079	7.0078	7.1971	7.2405
	24	8.3720	8.5332	8.8305	8.8971